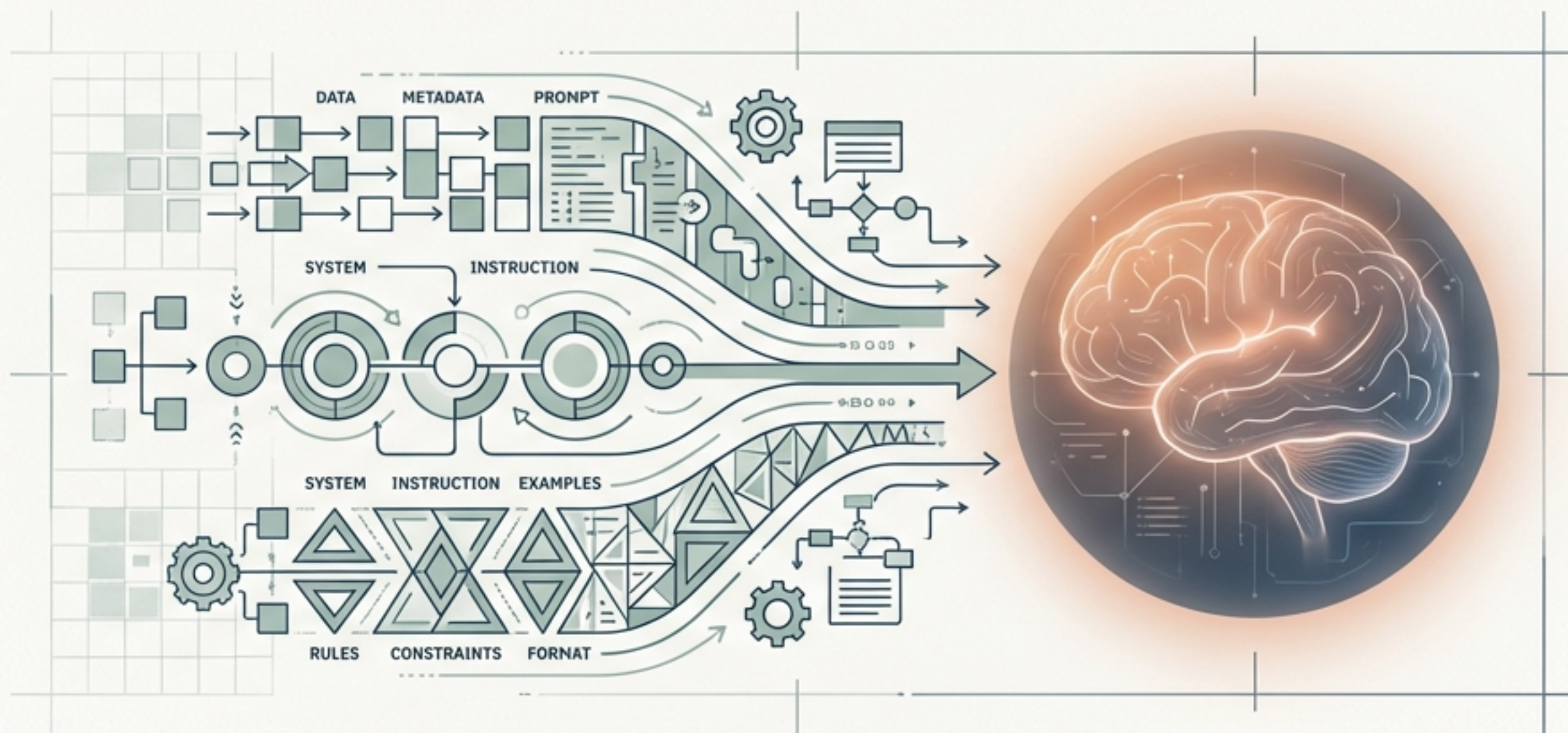


การสำรวจเกี่ยวกับ Context Engineering สำหรับ Large Language Models

กรอบการทำงานที่เป็นระบบเพื่อการออกแบบ จัดการ และปรับปรุงข้อมูลสำหรับ LLMs



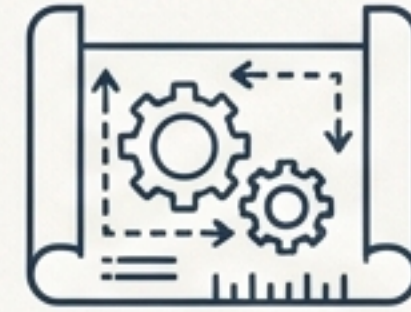
สรุปและเรียบเรียงจากผลงานวิจัยของ Lingrui Mei, Jiayu Yao, Yuyao Ge, และคณะ ซึ่งวิเคราะห์บทความวิจัยกว่า 1,400 ฉบับ

จากศิลปะสู่วิทยาศาสตร์: วัฒนาการที่เหนือกว่า Prompt Engineering



Prompt Engineering

- **คงที่ (Static):** คำสั่งเป็นข้อความที่ตายตัว
- **ทำด้วยมือ (Manual):** อาศัยการปรับแก้และลองผิดลองถูก
- **ไร้สถานะ (Stateless):** การโต้ตอบแต่ละครั้งไม่เชื่อมโยงกัน
- **เป้าหมาย:** ค้นหา prompt ที่ดีที่สุดสำหรับผลลัพธ์เดียว



Context Engineering

- **พลวัต (Dynamic):** บริบทถูกสร้างและปรับเปลี่ยนตามสถานการณ์
- **เป็นระบบ (Systematic):** มีกระบวนการทางวิศวกรรมที่ชัดเจน
- **มีสถานะ (Stateful):** จดจำและใช้ข้อมูลจากการโต้ตอบก่อนหน้านี้
- **เป้าหมาย:** การปรับปรุงระบบฟังก์ชันที่สร้างบริบทให้ได้ประสิทธิภาพสูงสุด

นิยามของศาสตร์แขนงใหม่: Context Engineering

ศาสตร์แห่งการปรับปรุงชุดข้อมูลสารสนเทศ (Information Payloads) สำหรับ LLMs
อย่างเป็นระบบ เพื่อเพิ่มประสิทธิภาพสูงสุด

$$C = A(c_instr, c_know, c_tools, c_mem, \dots)$$

Context ไม่ใช่แค่ 'prompt' ที่เป็นข้อความนิ่ง
แต่คือการประกอบกันอย่างมีพลวัตขององค์ประกอบต่างๆ

Context Engineering

องค์ประกอบพื้นฐาน
(Foundational
Components)

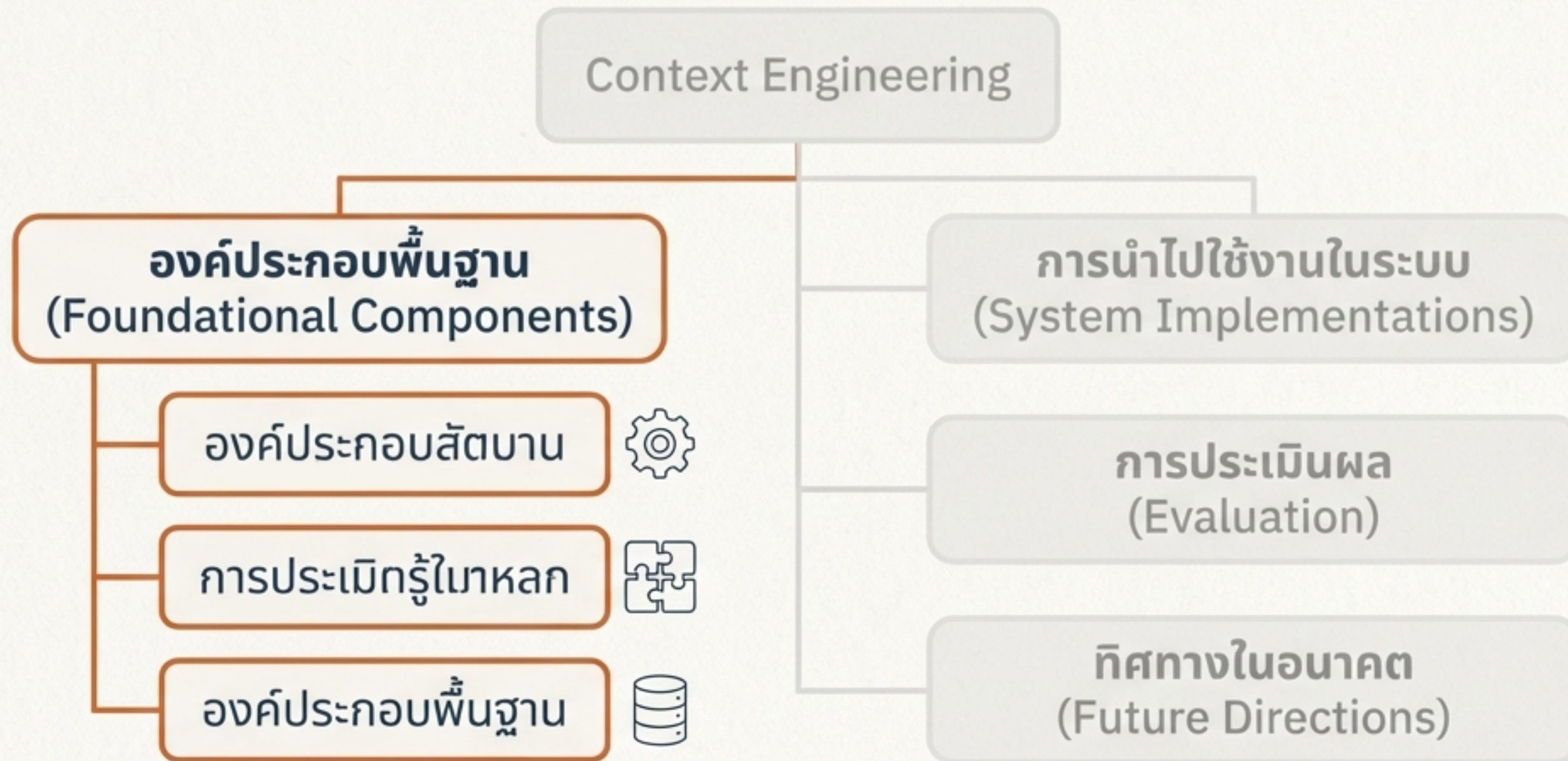
การนำไปใช้งานในระบบ
(System
Implementations)

การประเมินผล
(Evaluation)

ทิศทางในอนาคต
(Future Directions)

องค์ประกอบพื้นฐาน (Foundational Components)

ส่วนประกอบหลัก 3 ประการที่เป็นหัวใจของการสร้าง AI ที่ตระหนักรู้ในบริบท



1. การดึงข้อมูลและการสร้างบริบท (Context Retrieval & Generation)

การจัดการและสร้างข้อมูลที่เกี่ยวข้องเพื่อป้อนให้กับ LLMs



Prompt Engineering และการสร้างบริบท

การออกแบบคำสั่งและโครงสร้างการให้เหตุผลที่ซับซ้อน เช่น Chain-of-Thought (CoT) และ Tree-of-Thoughts (ToT) เพื่อนำทางกระบวนการคิดของโมเดล



การดึงความรู้จากภายนอก

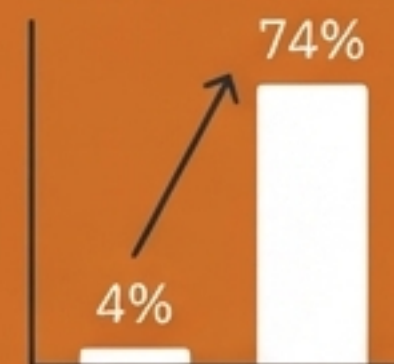
การเข้าถึงแหล่งข้อมูลภายนอกแบบไดนามิกเพื่อลดข้อจำกัดด้านความรู้ของโมเดล เช่น การใช้ Retrieval-Augmented Generation (RAG) หรือ Knowledge Graphs



การประกอบบริบทแบบไดนามิก

การจัดเรียงส่วนประกอบข้อมูลที่ได้มาทั้งหมดให้เป็นบริบทสุดท้ายที่สอดคล้องกันและเหมาะสมที่สุดสำหรับงานนั้นๆ

ข้อมูลสำคัญ: เทคนิค **Tree-of-Thoughts (ToT)** สามารถเพิ่มอัตราความสำเร็จในการแก้ปัญหา Game of 24 จาก **4%** เป็น **74%**



2. การประมวลผลบริบท (Context Processing)

การแปลงและปรับปรุงข้อมูลที่ได้มาเพื่อเพิ่มประโยชน์สูงสุด



การประมวลผลบริบทขนาดยาว

การรับมือกับความท้าทายจาก $O(n^2)$ complexity ของ self-attention ด้วยสถาปัตยกรรมใหม่ๆ เช่น Mamba และเทคนิคอย่าง FlashAttention



การปรับปรุงตนเองและการปรับตัว

กลไกที่ LLMs สามารถตรวจสอบและแก้ไขผลลัพธ์ของตนเองได้แบบวนซ้ำ เช่น การทำงาน Self-Refine และ Reflexion



การบูรณาการข้อมูลที่มีโครงสร้างและหลายรูปแบบ

การจัดการข้อมูลที่ซับซ้อนกว่าข้อความธรรมดา เช่น ตาราง, กราฟความรู้ (Knowledge Graphs) และข้อมูลหลายรูปแบบ (Multimodal)

ข้อมูลสำคัญ: กรอบการทำงาน Self-Refine ช่วยให้ GPT-4 มีประสิทธิภาพในการทำงานดีขึ้นประมาณ 20% ผ่านการตรวจสอบและแก้ไขผลลัพธ์ของตนเอง



3. การจัดการบริบท (Context Management)

การจัดระเบียบ จัดเก็บ และใช้งานข้อมูลอย่างมีประสิทธิภาพภายใต้ข้อจำกัด



ข้อจำกัดพื้นฐาน

การเอาชนะปัญหา 'lost-in-the-middle' ที่โมเดลลืมข้อมูลตรงกลาง และการจัดการ context window ที่มีขนาดจำกัด



ลำดับชั้นของหน่วยความจำ

การออกแบบสถาปัตยกรรมหน่วยความจำที่ซับซ้อน เลียนแบบระบบปฏิบัติการ เช่น MemGPT ที่มีการแบ่งหน้าหน่วยความจำ (paging) ไปยังที่จัดเก็บภายนอก



การบีบอัดบริบท

เทคนิคในการลดขนาดของบริบทเพื่อประหยัดทรัพยากรแต่ยังคงรักษาข้อมูลสำคัญไว้ เช่น StreamingLLM

ข้อมูลสำคัญ: เทคนิค StreamingLLM ช่วยให้สามารถประมวลผลลำดับข้อมูลที่ยาวไม่สิ้นสุดได้ โดยมีความเร็วเพิ่มขึ้นถึง **22.2 เท่า** เมื่อเทียบกับการใช้ sliding window แบบเดิม



22.2x Faster

การนำไปใช้งานในระบบ (System Implementations)

การผสานองค์ประกอบพื้นฐานเพื่อสร้างสถาปัตยกรรม AI อัจฉริยะที่ใช้งานได้จริง

องค์ประกอบพื้นฐาน (Foundational Components)



2. การประมวลผลบริบท (Context Processing)

การประมวลผลบริบท
เพื่อสร้างสถาปัตยกรรม
AI อัจฉริยะที่ใช้งานได้จริง



3. การจัดการบริบท (Context Management)

การผสานองค์ประกอบพื้นฐาน
เพื่อสร้างสถาปัตยกรรม AI
อัจฉริยะที่ใช้งานได้จริง

การนำไปใช้งานในระบบ (System Implementations)



4. สถาปัตยกรรมระบบ (System Architecture)

การผสานองค์ประกอบพื้นฐาน
เพื่อสร้างสถาปัตยกรรม AI
อัจฉริยะที่ใช้งานได้จริง



5. การประยุกต์ใช้ (Applications)

การผสานองค์ประกอบพื้นฐาน
เพื่อสร้างสถาปัตยกรรม AI
อัจฉริยะที่ใช้งานได้จริง



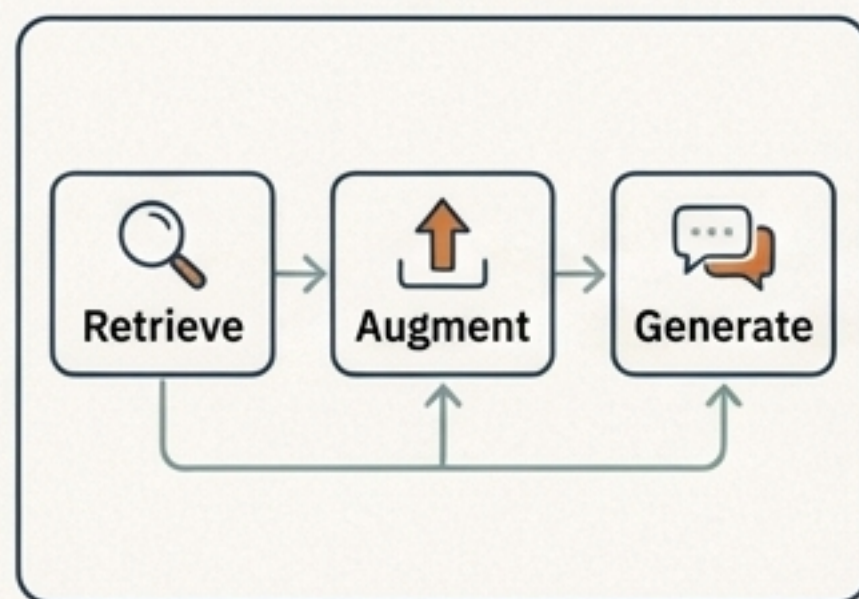
6. การปรับใช้และการดูแลรักษา (Deployment & Maintenance)

การปรับใช้และการดูแล
รักษา (Deployment &
Maintenance) ใช้งานได้จริง

1. Retrieval-Augmented Generation (RAG)

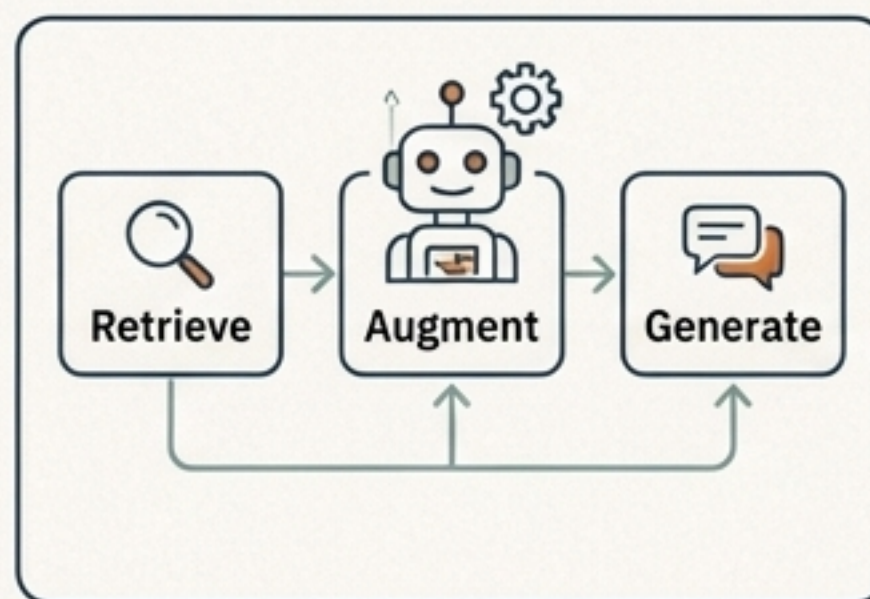
การเชื่อมช่องว่างระหว่างความรู้ในตัวแบบจำลองกับการเข้าถึงข้อมูลภายนอกแบบไดนามิก

Modular RAG



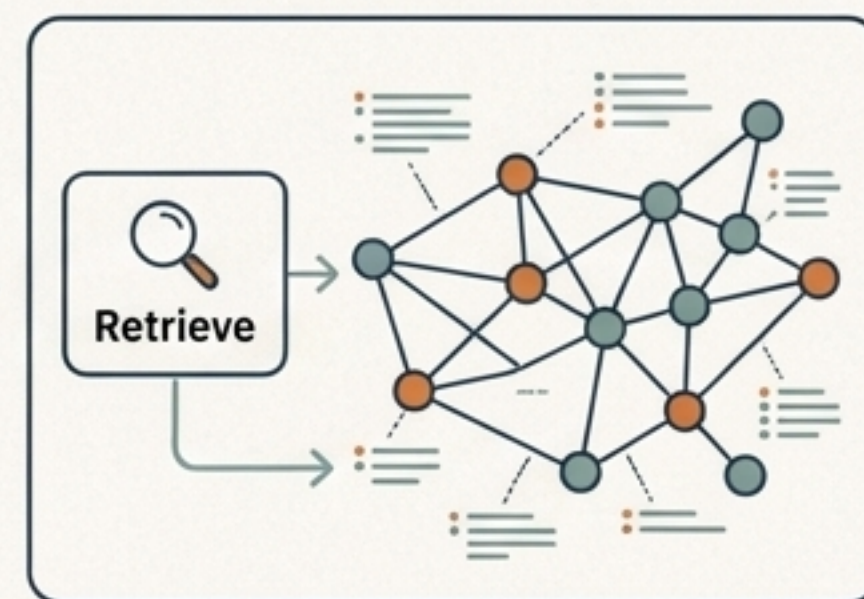
สถาปัตยกรรมแบบแยกส่วนที่ยืดหยุ่น สามารถปรับเปลี่ยนและกำหนดค่า องค์ประกอบการดึงข้อมูลและการ สร้างคำตอบได้อย่างอิสระ

Agentic RAG



การใช้ Agent ที่มีความสามารถในการ ให้เหตุผลและวางแผนมาทำหน้าที่เป็น "นักสืบข้อมูล" เพื่อค้นหาและวิเคราะห์ ข้อมูลอย่างชาญฉลาด

Graph-Enhanced RAG



การใช้ประโยชน์จาก Knowledge Graph เพื่อการดึงข้อมูลที่มีความ สัมพันธ์ซับซ้อน ทำให้การให้เหตุผล มีความแม่นยำและลึกซึ้งยิ่งขึ้น

ตัวอย่างจากงานวิจัย: Self-RAG, RAPTOR, HippoRAG, GraphRAG

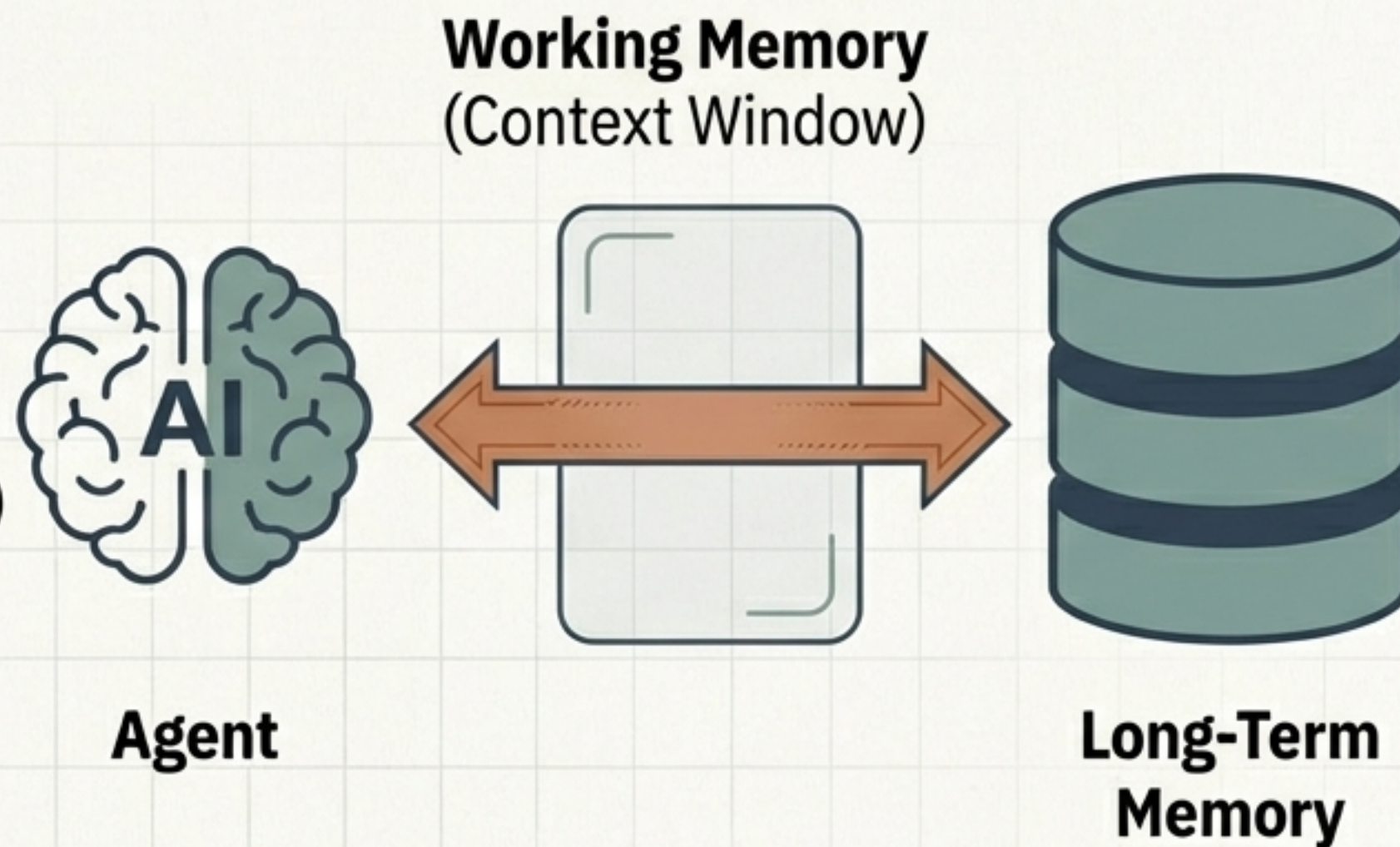
2. ระบบหน่วยความจำ (Memory Systems)

การสร้างปฏิสัมพันธ์ที่ต่อเนื่องและจดจำข้อมูลข้ามเซสชัน เพื่อให้ LLM สามารถเรียนรู้และปรับตัวได้

แนวคิดหลัก

สถาปัตยกรรมหน่วยความจำ

- **Short-Term Memory:** ข้อมูลที่อยู่ใน context window ปัจจุบัน
- **Long-Term Memory:** ข้อมูลที่จัดเก็บถาวรในแหล่งข้อมูลภายนอก (เช่น vector databases)



Memory-Enhanced Agents

Agent ที่สามารถเขียน อ่าน และปรับปรุงหน่วยความจำของตนเองได้ ทำให้สามารถเรียนรู้จากประสบการณ์ในอดีตเพื่อการตัดสินใจที่ดีขึ้น

ตัวอย่างจากงานวิจัย: MemGPT, MemoryBank, Self-Controlled Memory (SCM)

3 & 4. การใช้เครื่องมือ และระบบ Multi-Agent

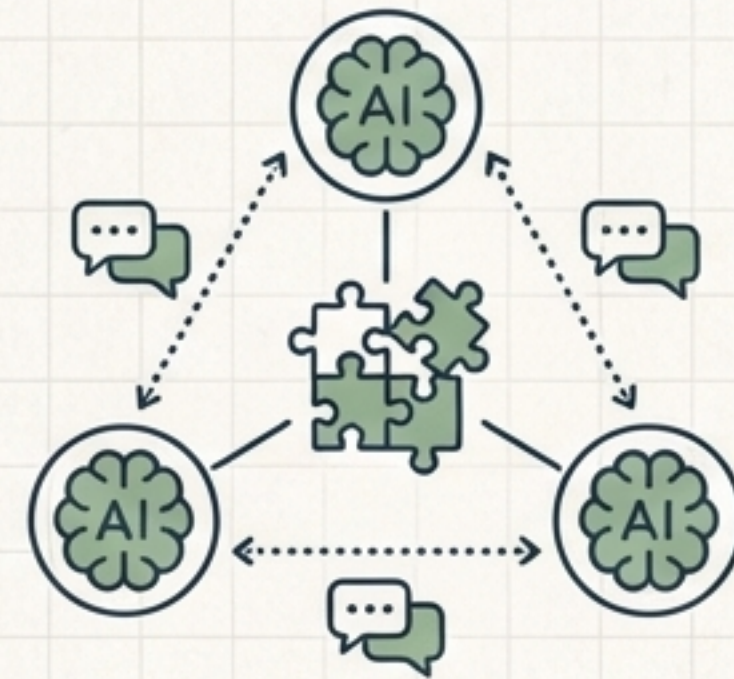
การใช้เหตุผลร่วมกับการใช้เครื่องมือ (Tool-Integrated Reasoning)



การเปลี่ยน LLM จาก 'ผู้สร้างข้อความ' ให้กลายเป็น 'ผู้มีปฏิสัมพันธ์กับโลก' โดยสามารถเรียกใช้เครื่องมือภายนอก (Function Calling) เช่น API, search engines หรือ code interpreters เพื่อทำงานที่อยู่นอกเหนือความสามารถของตัวเอง

ตัวอย่าง: Toolformer, ReAct, Gorilla, ToolLLM

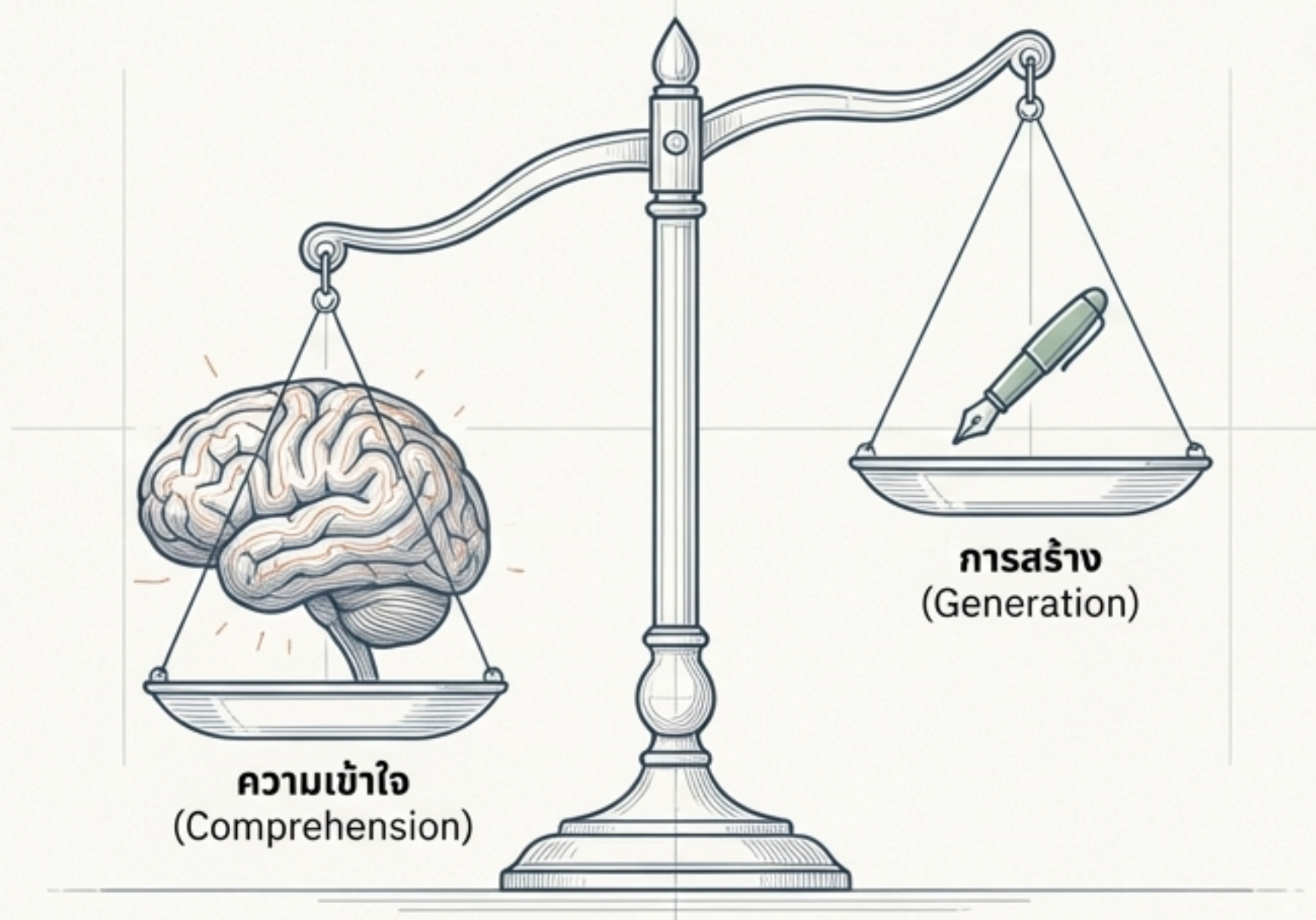
ระบบ Multi-Agent (Multi-Agent Systems)



การทำงานร่วมกันของ Agent อีกระหลายตัวเพื่อแก้ปัญหาที่ซับซ้อนเกินกว่าที่ Agent ตัวเดียวจะทำได้ โดยอาศัยโปรโตคอลการสื่อสาร (Communication Protocols) และกลไกการควบคุม (Orchestration Mechanisms) ที่มีประสิทธิภาพ

ตัวอย่าง: AutoGen, MetaGPT, CrewAI, CAMEL

ช่องว่างที่สำคัญที่สุด: ความไม่สมมาตรระหว่าง 'ความเข้าใจ' และ 'การสร้าง'



แบบจำลองในปัจจุบัน เมื่อเสริมด้วย Context Engineering ขึ้นสูง แสดงให้เห็นถึงความสามารถที่น่าทึ่งในการ **เข้าใจ** บริบทที่ซับซ้อน แต่กลับมีข้อจำกัดที่ชัดเจนในการ **สร้าง** ผลลัพธ์ที่มีความซับซ้อนและยาวในระดับเดียวกัน

ทิศทางในอนาคตและความท้าทาย



การวิจัยพื้นฐาน (Foundational Research)

- ทฤษฎีและกรอบการทำงานที่เป็นหนึ่งเดียว
- กฎการขยายขนาดและประสิทธิภาพการคำนวณ
- การบูรณาการข้อมูลหลายรูปแบบ (Multi-modal) และเชิงโครงสร้าง (Graph)



นวัตกรรมทางเทคนิค (Technical Innovation)

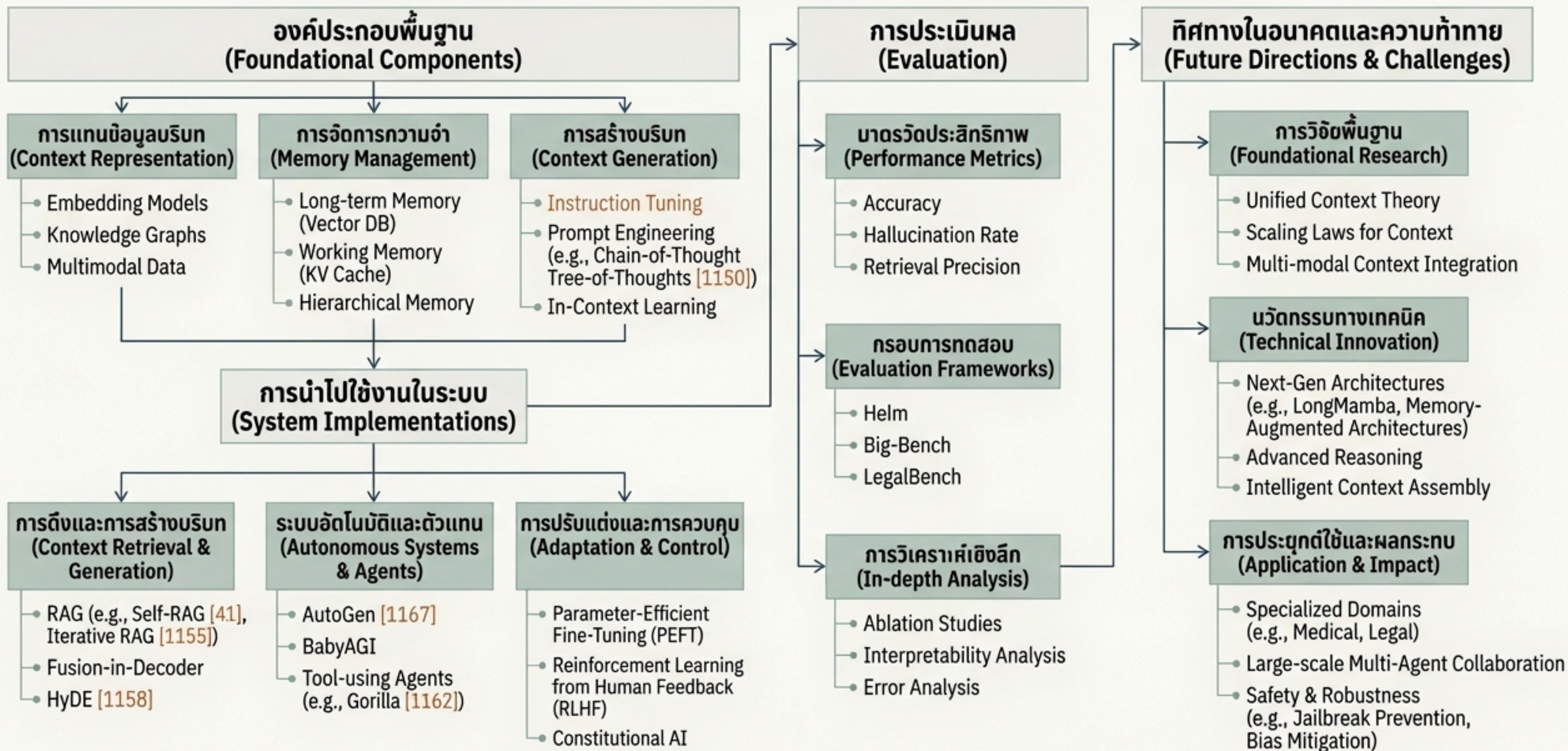
- สถาปัตยกรรมยุคถัดไป (เช่น LongMamba, Memory-Augmented Architectures)
- การให้เหตุผลและการวางแผนขั้นสูง
- การประกอบและปรับปรุงบริบทอัจฉริยะ (Intelligent Context Assembly)



การประยุกต์ใช้และผลกระทบ (Application & Impact)

- ความเชี่ยวชาญเฉพาะทาง (เช่น การแพทย์, กฎหมาย)
- การประสานงานของ Multi-Agent ขนาดใหญ่
- ความปลอดภัย (Safety), ความมั่นคง (Security), และความน่าเชื่อถือ (Robustness)

แผนผัง Context Engineering ฉบับสมบูรณ์



Context Engineering:

ศาสตร์แห่งการสร้าง AI ที่ตระหนักรู้ในบริบท

การเปลี่ยนผ่านจาก “ศิลปะ” ของการออกแบบ prompt ไปสู่ "วิทยาศาสตร์" ของการจัดการสารสนเทศอย่างเป็นระบบ คือหัวใจสำคัญของการพัฒนาระบบ AI ยุคถัดไป

Paper Title

A Survey of Context Engineering for Large Language Models

Primary Authors

Lingrui Mei, Jiayu Yao, Yuyao Ge, et al.

Code & Resources



<https://github.com/Meirtz/Awesome-Context-Engineering>